

A2Q3

Lily Li

2025-06-11

Q3

```
library(cancensus)
```

```
## Census data is currently stored temporarily.
```

```
##
```

```
## In order to speed up performance, reduce API quota usage, and reduce unnecessary network calls, please
```

```
## This will add your cache directory as environment variable to your .Renviron to be used across sessions
```

```
#all_vecs = list_census_vectors("CA21")
```

```
# Total English          v_CA21_2209
```

```
# Total French           v_CA21_2212
```

```
# Full time Full year    v_CA21_6525
```

```
# Average wks worked     v_CA21_6531
```

```
# Median after-tax inc   v_CA21_986
```

```
# Median Total inc       v_CA21_983
```

```
all_regions = list_census_regions("CA21") # 2021 census
```

```
## Querying CensusMapper API for regions data...
```

```
csd_all = list(CSD = all_regions$region)
```

```
all_regions = data.frame(all_regions)
```

```
thislevel = "CSD"
```

```
census_data = get_census(dataset = "CA21",  
                          regions = csd_all,  
                          vectors = c("v_CA21_2209",  
                                       "v_CA21_2212",  
                                       "v_CA21_983",  
                                       "v_CA21_986",  
                                       "v_CA21_6525",  
                                       "v_CA21_6531"),  
                          level = thislevel)
```

```
## Census data is currently stored temporarily.
```

```
##
```

```
## In order to speed up performance, reduce API quota usage, and reduce unnecessary network calls, please
```

```
## This will add your cache directory as environment variable to your .Renviron to be used across sessions
```

```
## Census data is currently stored temporarily.
##
## In order to speed up performance, reduce API quota usage, and reduce unnecessary network calls, please
## This will add your cache directory as environment variable to your .Renviron to be used across sessions
```

```
## Querying CensusMapper API...
```

```
## Downloading: 32 kB      Downloading: 32 kB      Downloading: 81 kB      Downloading: 81 kB      Downloading: 81 kB
```

```
names(census_data)[ncol(census_data) - 5 ] = "English_Speakers"
names(census_data)[ncol(census_data) - 4 ] = "French_Speakers"
names(census_data)[ncol(census_data) - 3 ] = "Median_Before_Tax"
names(census_data)[ncol(census_data) - 2 ] = "Median_After_Tax"
names(census_data)[ncol(census_data) - 1 ] = "FTime_FYear_Workers"
names(census_data)[ncol(census_data) - 0 ] = "Avg_Weeks_Worked"

census_data = data.frame(census_data)
census_data = subset(census_data, Population > 0 & !is.na(Avg_Weeks_Worked)
                     & !is.na(Median_After_Tax))
filename = paste0("Stat341_S25_A2_Census_Data_",thislevel, ".csv")
write.csv(census_data, filename, row.names=FALSE)
```

(a)

```
library(quantreg)
```

```
## Loading required package: SparseM
```

```
# for testing, i'll use Median_Before_Tax to predict Median_After_Tax
my_data <- data.frame(
  x = census_data$Median_Before_Tax,
  y = census_data$Median_After_Tax
)
```

```
rho_lm_fn <- function(x, data) {
  alpha <- x[1]
  beta <- x[2]
  y_hat <- alpha + beta * data[[1]]
  residuals <- data[[2]] - y_hat
  sum(residuals^2)
}
```

(b)

```
opt_lm <- optim(c(0, 0), rho_lm_fn, data = my_data)
print(opt_lm$par)
```

```
## [1] 0.2499038 0.9032800
```

```
lm_model <- lm(y ~ x, data = my_data)
print(coef(lm_model))
```

```
## (Intercept)          x
## 5261.2417354    0.7674508
```

(c)

```
rho_qr_fn <- function(x, tau, data) {
  alpha <- x[1]
  beta <- x[2]
  residuals <- data[[2]] - (alpha + beta * data[[1]])
  sum(ifelse(residuals >= 0, tau * residuals, (tau - 1) * residuals))
}
```

(d)

```
taus <- c(0.1, 0.25, 0.5, 0.75, 0.9)

results <- data.frame(
  Tau = taus,
  Alpha_optim = NA, Beta_optim = NA,
  Alpha_rq = NA, Beta_rq = NA
)

for (i in seq_along(taus)) {
  tau <- taus[i]

  opt_result <- optim(c(0, 0), rho_qr_fn, tau = tau, data = my_data)
  rq_model <- rq(y ~ x, tau = tau, data = my_data)

  results$Alpha_optim[i] <- opt_result$par[1]
  results$Beta_optim[i] <- opt_result$par[2]
  results$Alpha_rq[i] <- coef(rq_model)[1]
  results$Beta_rq[i] <- coef(rq_model)[2]
}

print(results)
```

##	Tau	Alpha_optim	Beta_optim	Alpha_rq	Beta_rq
## 1	0.10	0.2595289	0.8602697	5538.462	0.7307692
## 2	0.25	0.2193677	0.8837227	5563.636	0.7424242
## 3	0.50	5269.2358300	0.7673489	5265.116	0.7674419
## 4	0.75	0.2489478	0.9363011	5189.189	0.7837838
## 5	0.90	0.3822172	0.9753824	5120.000	0.8000000

(e)

The slope estimates from both `optim()` and `rq()` are generally close, indicating the generalized optimizer approximates the quantile regression well. However, there is a significant discrepancy in the intercept estimates, likely due to scaling or sign-handling in the asymmetric loss function. This suggests that while `optim()` is flexible, it may require careful setup, and domain-specific functions like `rq()` are more stable and robust for this type of modeling.